Unisphere

A Full-Stack Teaching Handbook

Learning real web development by studying one real application — the one already running in your workspace.

Written for a complete beginner. Nothing is assumed. Every technology is explained before it is used, and every explanation points at a real file in your repository.

Before we begin: how to use this handbook

This handbook is not documentation. It is a course. It follows three rules.

Rule 1 — Nothing is assumed

Every technology gets five questions answered before you see any code that uses it: what is it, why do we need it, what problem does it solve, what would happen without it, and where does this project use it. If a word appears that you have not met, it is defined on the spot.

Rule 2 — Architecture before files

Beginners usually start by opening a random file and get lost, because a single file makes no sense on its own. So we start from the outside: what the whole system looks like, how a click travels through it, and only then what individual files contain.

Rule 3 — Every concept is anchored in your real code

There are no invented examples of "a typical app". Every snippet in this handbook was copied out of your repository. Filenames and line contents are real. If a snippet is shortened, it says so.

A note on honesty about difficulty

Some parts of this project are genuinely hard — WebRTC video calling is a topic that professional engineers find difficult. This handbook will never tell you "don't worry about this". Instead it tells you: *you do not need to master this today, and here is exactly what it does.* Each major area is labelled:

- **MUST UNDERSTAND NOW** — core concepts; everything else is built on them.
- **SHOULD UNDERSTAND SOON** — important, but it can wait a few weeks.
- **CAN TREAT AS A LIBRARY** — complex machinery you can use correctly long before you could write it from scratch. Professionals do this every day.

PART 1 — What this application does

Unisphere is a web application for undergraduate university students. In one sentence: **it is LinkedIn for undergraduates, with the spontaneous video discovery of OmeTV.**

It exists because a student's real network is accidentally tiny — it is whoever happens to sit in their classroom. Unisphere's tagline is the product thesis: *"Your Campus Is Bigger Than Your Campus."*

The eighteen things a user can actually do

Area	
Authentication	Sign up, with student details, log in, log out, stay lo
Profile	Create and edit a profile: college, degree, branch, year, skills, interests, links, reputa
Home feed	Post text/image/video, like, comment, share, bookm
Stories	24-hour ephemeral video/text stories, viewer, reactions,
Dashboard	Personalised greeting, recommendations, upcomin
Discover	Filter students by college, degree, branch, year, ci
Meet Someone	Pick intents, join a queue, get matched with a live s
Video calling	Real WebRTC 1-to-1 call: camera, mic, screen sha
Messaging	Conversation list, unread badges, thread history, s
Projects	Create projects, list required roles, request to join,
Meetings	Publish weekly availability, book a slot, track meeti
Communities	College hubs with rosters, events, follower counts
Connections	Send / accept / reject / remove connection request
Notifications	Likes, comments, requests, bookings, calls, match
Reputation	Five-dimension peer endorsement with anti-abuse
Safety	Report, block users, privacy toggles
Admin	Metrics, moderation queue, suspend users, delete
Search	One search box across students, colleges, project

Scale of the codebase: **17 backend routers, 18 frontend pages, 22 database collections, roughly 10,400 lines of code.** That is a small-to-medium real product, not a toy.

PART 2 — The technology stack, explained from zero

Before the list, one idea that makes the whole list make sense.

The single most important idea in web development

A web application is always **two programs talking to each other over the internet**.

```
PROGRAM 1: the FRONTEND                PROGRAM 2: the BACKEND
runs INSIDE the user's browser         runs on a SERVER you control
(Chrome on a phone or laptop)          (a computer in a data centre)

It can:  draw the screen               It can:  read/write the database
         react to clicks                        check passwords
         send requests                          enforce the rules
It CANNOT be trusted                   It IS the source of truth
```

Why can't the frontend be trusted? Because it runs on the *user's* machine. Anyone can open the browser's developer tools and change it. So the frontend is only ever a **display and input layer**. Every decision that matters — is this password correct, may this person delete this post — must be made by the backend.

> *Remember this sentence: the frontend asks, the backend decides. Almost every security bug in beginner projects is a violation of it.*

2.1 The languages

HTML

- **What is it?** The language that describes the *structure* of a page: headings, buttons, images, inputs.
- **Why do we need it?** A browser cannot show anything without structure to render.
- **What problem does it solve?** It separates content and meaning from appearance.
- **Without it?** You would have no page at all.
- **Where in Unisphere?** You will barely see raw HTML here, because React generates it. The one real HTML file is the shell the browser loads first.
- **Files to open:** `frontend/index.html`

CSS

- **What is it?** The language that describes *appearance*: colour, size, spacing, layout, animation.
- **Why do we need it?** Unstyled HTML is black text on white, unusable on a phone.
- **What problem does it solve?** It lets one structure look different on desktop and mobile, and lets you restyle without touching logic.
- **Without it?** Your app would look like a 1995 document.
- **Where in Unisphere?** All theme colours, fonts and the signature glow/aura effects live in one stylesheet as CSS variables.
- **Files to open:** `frontend/src/index.css`

JavaScript

- **What is it?** The only programming language browsers can run. It makes pages *do* things: respond to clicks, fetch data, update the screen.
- **Why do we need it?** HTML and CSS are static descriptions. Nothing is interactive without JS.
- **What problem does it solve?** It turns a document into an application.
- **Without it?** Every button in Unisphere would be decoration.
- **Where in Unisphere?** Indirectly everywhere — because TypeScript (next) compiles down to JavaScript, and that is what actually runs in the browser.
- **Files to open:** `every .tsx / .ts file, after compilation`

TypeScript

- **What is it?** JavaScript plus a type system. You declare what shape your data has, and a compiler checks you never break your own promise. Types vanish at runtime; they exist only to catch your mistakes while you write.
- **Why do we need it?** JavaScript will happily let you write post.titel and fail silently at 2 a.m. in production.
- **What problem does it solve?** It converts a whole class of runtime crashes into errors on your screen while typing.
- **Without it?** Renaming one field in the backend would break five pages, and you would find out from a user, not a compiler.
- **Where in Unisphere?** The entire frontend. Every response shape from the backend has a matching TypeScript interface.
- **Files to open:** `frontend/src/lib/types.ts (435 lines of these shapes)`

Python

- **What is it?** A general-purpose programming language, very readable, dominant in backend and data work.
- **Why do we need it?** The backend needs a language too, and it does not have to be JavaScript.
- **What problem does it solve?** It gives fast development with a huge ecosystem of libraries.
- **Without it?** No backend.
- **Where in Unisphere?** The entire server side.
- **Files to open:** `everything under backend/`

Your brief suggested a Node.js backend. This project uses Python + FastAPI instead, because that is the stack the workspace is built on. Every concept you learn here — routes, middleware, models, JWT — transfers to Node/Express almost one-to-one. The ideas are the skill; the language is a detail.

2.2 The frontend libraries

React

- **What is it?** A JavaScript library for building user interfaces out of **components** — small reusable pieces that each own a bit of screen. You describe what the screen should look like for a given set of data; React works out how to update the real page.
- **Why do we need it?** Updating a page by hand ("find this element, change this text, insert this row") becomes unmanageable past a few screens.

- **What problem does it solve?** It removes manual DOM manipulation and gives you reusable, composable UI.
- **Without it?** You would write thousands of lines of fragile "find element, change it" code.
- **Where in Unisphere?** All 18 pages and every component.
- **Files to open:** `frontend/src/pages/*.tsx, frontend/src/components/*.tsx`

Vite

- **What is it?** The build tool and development server. It serves your code to the browser instantly while you develop, and bundles it into optimised files for production.
- **Why do we need it?** Browsers cannot read .tsx files or import npm packages directly.
- **What problem does it solve?** Instant startup, hot reload (save a file, the screen updates without losing state), and an optimised production build.
- **Without it?** You would refresh manually and ship huge slow bundles.
- **Where in Unisphere?** It runs your frontend on port 3000, and forwards every /api request to the backend — the trick that lets frontend code use plain relative URLs.
- **Files to open:** `frontend/vite.config.ts`

Tailwind CSS

- **What is it?** A styling system where you compose small utility classes directly in your markup (p-4 = padding, flex = flex layout, text-violet-400 = colour) instead of writing separate CSS rules.
- **Why do we need it?** In big projects, hand-written CSS files grow into a tangle nobody dares delete from.
- **What problem does it solve?** Styles live next to the markup they affect, so deleting a component deletes its styles too. Dead CSS becomes impossible.
- **Without it?** You would maintain thousands of lines of CSS with names like .card-inner-2.
- **Where in Unisphere?** Every component's className attribute.
- **Files to open:** `any .tsx file; theme tokens in frontend/src/index.css`

shadcn/ui

- **What is it?** A collection of ready-made, accessible React components (button, dialog, dropdown, input, badge...) that are **copied into your project** rather than installed as a black box.
- **Why do we need it?** Building a keyboard-accessible dropdown or modal correctly is genuinely hard and takes days.
- **What problem does it solve?** Professional, accessible building blocks you can still open and edit, because the code is yours.
- **Without it?** You would ship inaccessible custom widgets, or spend weeks rebuilding solved problems.
- **Where in Unisphere?** Buttons, dialogs, dropdown menus, inputs, badges throughout the UI.
- **Files to open:** `frontend/src/components/ui/`

TanStack Query

- **What is it?** A data-fetching library. You tell it "this screen needs the posts" and it handles caching, loading flags, errors, retries, de-duplication and refetching.

- **Why do we need it?** Every screen otherwise re-implements loading spinners, error states and cache invalidation, slightly differently, forever.
- **What problem does it solve?** It removes the most repetitive and bug-prone code in any frontend: server state management.
- **Without it?** You would fetch inside useEffect, double-fetch on every render, and show stale data after saving.
- **Where in Unisphere?** Every read (useQuery) and every write (useMutation) in the app.
- **Files to open:** `frontend/src/lib/queryClient.ts and every page`

React Router

- **What is it?** Turns URLs into screens inside a single-page app, so /feed and /profile/123 render different components without a full page reload.
- **Why do we need it?** A single-page app has one HTML file; without a router every URL would show the same thing.
- **What problem does it solve?** Real, shareable, bookmarkable URLs plus instant navigation.
- **Without it?** One screen, no back button, no links you can share.
- **Where in Unisphere?** The route table for all 18 pages.
- **Files to open:** `frontend/src/App.tsx`

sonner

- **What is it?** A toast-notification library — the small messages that slide in saying "Post created".
- **Why do we need it?** Users need confirmation that an action worked, without a blocking alert box.
- **What problem does it solve?** Non-intrusive feedback.
- **Without it?** Users click a button, nothing visible happens, and they click it four more times.
- **Where in Unisphere?** Success/error feedback across the app, anchored bottom-right so it never covers the notification bell.
- **Files to open:** `frontend/src/App.tsx (the Toaster), toast() calls in components`

2.3 The backend libraries

FastAPI

- **What is it?** A Python web framework: it turns Python functions into HTTP endpoints (URLs the frontend can call), validates incoming data, and generates interactive API documentation automatically.
- **Why do we need it?** Raw HTTP handling is tedious and easy to get wrong.
- **What problem does it solve?** You write a normal Python function, declare the data shape, and get validation, error responses and docs for free.
- **Without it?** You would hand-parse requests and hand-write validation for all 100+ endpoints.
- **Where in Unisphere?** Every endpoint in the app.
- **Files to open:** `backend/server.py, backend/routers/*.py`

Uvicorn

- **What is it?** The web server process that actually listens on a network port and runs your FastAPI app.

- **Why do we need it?** A framework is just code; something has to hold the socket open and accept connections.
- **What problem does it solve?** It handles the network layer, and in development it restarts automatically when you save a file.
- **Without it?** Your API would never be reachable.
- **Where in Unisphere?** Runs your backend on port 8001, supervised so it restarts if it crashes.
- **Files to open:** `managed by supervisor, program name 'backend'`

Pydantic v2

- **What is it?** A data-validation library. You declare a class describing the fields and types you expect; Pydantic enforces it and rejects anything else with a clear error.
- **Why do we need it?** Data arriving from the internet is *never* trustworthy — it may be missing fields, wrong types, or hostile.
- **What problem does it solve?** Validation at the boundary. Bad data is rejected before it can reach your logic or database.
- **Without it?** You would write if not isinstance(...) checks by hand, forget some, and store corrupt records.
- **Where in Unisphere?** Every request body and response shape.
- **Files to open:** `backend/models/schemas.py`

MongoDB

- **What is it?** A database: the program that stores your data permanently on disk. MongoDB is a *document* database — it stores JSON-like documents inside named collections, instead of fixed rows in tables.
- **Why do we need it?** Program memory disappears the moment the process restarts. Users expect their posts to still be there tomorrow.
- **What problem does it solve?** Durable storage plus fast querying by index.
- **Without it?** Every post, message and account would vanish on restart.
- **Where in Unisphere?** All 22 collections of application data.
- **Files to open:** `backend/lib/db.py`

Motor

- **What is it?** The asynchronous MongoDB driver for Python — the library your code uses to send queries to MongoDB without blocking the server while it waits.
- **Why do we need it?** A database query takes milliseconds, and during those milliseconds a blocking server can serve nobody else.
- **What problem does it solve?** It lets one server handle many simultaneous users while queries are in flight.
- **Without it?** Your API would serve requests strictly one at a time and crawl under load.
- **Where in Unisphere?** Every database call: await db.posts.find_one(...).
- **Files to open:** `backend/lib/db.py exports the shared db handle`

passlib + bcrypt

- **What is it?** A password-hashing library. Hashing turns a password into a fixed-length scramble that cannot be reversed. bcrypt is deliberately slow, which makes guessing attacks expensive.
- **Why do we need it?** You must be able to check a password without ever being able to read it.
- **What problem does it solve?** If your database leaks, attackers still do not have your users' passwords.
- **Without it?** A single database leak would expose every user's real password — and, because people reuse passwords, their email and bank logins too.
- **Where in Unisphere?** Signup hashes; login verifies.
- **Files to open:** `backend/lib/auth.py`

PyJWT

- **What is it?** Creates and verifies JSON Web Tokens: a small signed string that says "this is user X" and cannot be tampered with without breaking the signature.
- **Why do we need it?** HTTP is stateless — the server forgets you between requests. Something must prove who you are on each one.
- **What problem does it solve?** Stateless authentication: no session table, no server memory of logins.
- **Without it?** Users would have to send their password with every single request.
- **Where in Unisphere?** Issued at signup/login, verified on every protected request.
- **Files to open:** `backend/lib/auth.py`

python-dotenv

- **What is it?** Loads configuration from a .env file into the program's environment variables.
- **Why do we need it?** Secrets (database URL, JWT secret, API keys) must never be written into source code that gets committed or shipped to browsers.
- **What problem does it solve?** It separates configuration from code, so the same code runs in development and production with different settings.
- **Without it?** Your secrets would be in your Git history forever, readable by anyone with repo access.
- **Where in Unisphere?** MONGO_URL, DB_NAME, CORS_ORIGINS, JWT_SECRET.
- **Files to open:** `backend/.env, read via os.environ`

2.4 The browser platform features

WebRTC

- **What is it?** A set of capabilities built into every modern browser that lets two browsers send audio and video **directly to each other**, without the media passing through your server.
- **Why do we need it?** Video is enormous. Relaying every call through your server would cost a fortune and add lag.
- **What problem does it solve?** Real-time peer-to-peer audio/video with low latency.
- **Without it?** You would need a paid media server, or your "video call" would be a fake screen.
- **Where in Unisphere?** Real 1-to-1 calls and the Meet Someone matches.
- **Files to open:** `frontend/src/lib/useWebRTC.ts, backend/routers/calls_router.py, meet_router.py`

localStorage

- **What is it?** A small key-value store inside the browser that survives page reloads and browser restarts.
- **Why do we need it?** Without it, refreshing the page would log the user out, because JavaScript memory is wiped.
- **What problem does it solve?** Session persistence on the client.
- **Without it?** You would log in again after every refresh.
- **Where in Unisphere?** The login token is stored under the key unisphere_auth_token.
- **Files to open:** `frontend/src/lib/session.ts, api.ts, auth-context.tsx`

HTTP polling (used here instead of WebSockets)

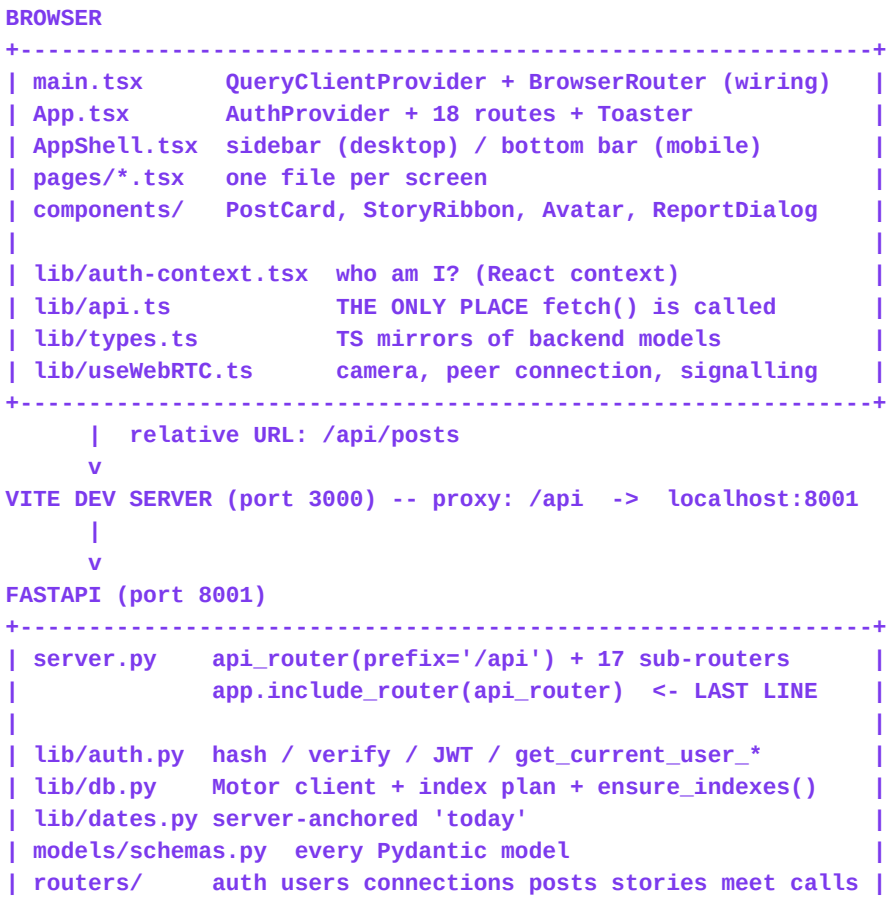
- **What is it?** Polling means the frontend repeatedly asks "anything new?" on a timer. WebSockets instead keep one connection permanently open so the server can push instantly.
- **Why do we need it?** Chat and call signalling need to feel live.
- **What problem does it solve?** Polling achieves near-real-time with plain HTTP and no extra infrastructure.
- **Without it?** Nothing — but it costs more requests than WebSockets, and updates arrive up to one interval late.
- **Where in Unisphere?** Chat refresh, notification refresh, and WebRTC signalling exchange every 2.5 seconds.
- **Files to open:** `frontend/src/lib/useWebRTC.ts line ~179: setInterval(poll, 2500)`

This is a deliberate, declared trade-off, not an oversight. Polling is simpler and adequate at this scale; WebSockets are the documented upgrade path, and because all traffic already flows through one fetch layer, swapping them in later touches few files. Part 7 LEVEL 9 covers this.

PART 3 — High-level architecture

The simple version

The real version, with the actual pieces of your project



```
|              messages projects meetings colleges reputation  |
|              notifications safety admin search ai            |
+--------------------------------------------------------------+
      |  await db.<collection>...
      v
MONGODB (in-pod, 22 collections, indexed at startup)

SIDE CHANNEL (media does NOT touch the server):
Browser A  <===== audio/video, peer-to-peer (WebRTC) =====>  Browser B
           the server only relays offer / answer / ICE
```

How one click travels: the Like button, for real

This is the single most useful diagram in the handbook. Every trace in Appendix B has this shape.

```
 1  User taps the heart on a post
      |
 2  PostCard.tsx: onClick={() => likeMutation.mutate()}
      |
 3  useMutation calls apiPost(`/posts/${post.id}/like`)
      |
 4  lib/api.ts attaches Authorization: Bearer <token>, calls fetch()
      |
 5  Vite proxy forwards /api/posts/<id>/like  ->  :8001
      |
 6  FastAPI matches @router.post('/{post_id}/like') in posts_router.py
      |
 7  Depends(get_current_user_required) decodes the JWT, loads the user,
    rejects suspended accounts, raises 401 if anything is wrong
      |
 8  Handler reads the post          -> 404 if it does not exist
      |
 9  Handler checks post_likes for (post_id, user_id)
      |         already liked?                  not liked yet?
      |         delete the like                 insert the like
      |         $inc likes_count by -1          $inc likes_count by +1
      |                                         insert a notification
      |                                         for the post's author
      |
10  Returns {"liked": true, "likes_count": 8}
      |
11  onSuccess -> queryClient.invalidateQueries({queryKey:['posts']})
      |
12  TanStack Query refetches /api/posts, React re-renders
      |
13  The heart is filled and the count reads 8
```

Notice three things a beginner would not guess:

- **The like is a separate document, not a field on the post.** post_likes holds one document per (post, user) pair, with a unique index — that is what makes double-liking impossible at the database level, not just in the UI.
- **likes_count is a duplicated counter.** The true answer is "count the documents in post_likes", but counting on every read is slow, so the count is stored on the post and adjusted with $inc. This is called *denormalisation*: trading a little duplication for a lot of speed.
- **The frontend does not calculate the new count.** It throws its cache away and asks the server again. The server stays the one source of truth, so two devices can never disagree.

Step 11 is where most beginner apps break. They update local state by hand, the server disagrees, and the UI shows a number that does not exist. Invalidate, don't guess.

PART 4 — Project folder and file map

This is the real tree in your workspace. Nothing here is invented.

```
/app
+-- backend/                        the server (Python)
|   +-- server.py                   entry point: builds the app, mounts 17 routers
|   +-- seed.py                     fills the DB with demo students and content
|   +-- requirements.txt            pinned Python dependencies
|   +-- .env                        SECRETS: MONGO_URL, DB_NAME, JWT_SECRET (never committed)
|   +-- lib/
|   |   +-- db.py                   Mongo client + the index plan + ensure_indexes()
|   |   +-- auth.py                 hashing, JWT, get_current_user_* dependencies
|   |   +-- dates.py                server-side 'today' helper
|   +-- models/
|   |   +-- schemas.py              EVERY Pydantic model (request + response shapes)
|   +-- routers/                    one file per feature area
|       +-- auth_router.py            signup, login, logout, me, reset-password
|       +-- users_router.py           profiles, edit, discovery filters
|       +-- connections_router.py     request / accept / reject / remove
|       +-- posts_router.py           feed CRUD, like, comment, bookmark, report
|       +-- stories_router.py         24h stories, views, reactions
|       +-- messages_router.py        conversations + messages
|       +-- projects_router.py        projects + join requests
|       +-- meetings_router.py        availability + bookings
|       +-- meet_router.py            Meet Someone queue + matching + signalling
|       +-- calls_router.py           1-to-1 call sessions + signalling
|       +-- colleges_router.py        community hubs, follows, events
|       +-- reputation_router.py      5-dimension endorsements
|       +-- notifications_router.py   notification feed
|       +-- safety_router.py          reports + blocks + privacy
|       +-- admin_router.py           metrics + moderation actions
|       +-- search_router.py          global multi-entity search
|       +-- ai_router.py              AI bio / pitch / icebreaker helpers
|
+-- frontend/                       the browser app (TypeScript)
|   +-- index.html                  the single HTML file the browser loads
|   +-- vite.config.ts              dev server + the /api -> :8001 proxy
|   +-- package.json                JS dependencies and scripts (yarn typecheck)
|   +-- public/                     files served as-is (the two decks live here)
|   +-- src/
|       +-- main.tsx                mounts React; QueryClientProvider + BrowserRouter
|       +-- App.tsx                 route table for all 18 pages + Toaster
|       +-- index.css               THEME: colour tokens, fonts, signature effects
|       +-- pages/                  one file per screen (18)
|       |   +-- Landing.tsx Login.tsx Signup.tsx Dashboard.tsx Feed.tsx
|       |   +-- Discover.tsx Profile.tsx Messages.tsx MeetSomeone.tsx
|       |   +-- CallRoom.tsx Projects.tsx Meetings.tsx Communities.tsx
|       |   +-- Connections.tsx Notifications.tsx Search.tsx Settings.tsx Admin.tsx
|       +-- components/
|       |   +-- AppShell.tsx        nav shell: sidebar on desktop, bottom bar on mobile
|       |   +-- PostCard.tsx        one post + like/comment/share/bookmark/report
|       |   +-- StoryRibbon.tsx     the stories carousel and viewer
|       |   +-- Avatar.tsx          profile picture with story aura
|       |   +-- ReportDialog.tsx    the reusable report modal
|       |   +-- States.tsx          shared loading / error / empty states
|       |   +-- ui/                 shadcn/ui primitives (button, dialog, input, ...)
|       +-- lib/
|           +-- api.ts              the typed fetch layer (the only fetch in the app)
```

```
|           +-- types.ts          TypeScript mirrors of the Pydantic models
|           +-- auth-context.tsx  logged-in user, login/signup/logout
|           +-- session.ts        the localStorage token key
|           +-- queryClient.ts    TanStack Query configuration
|           +-- useWebRTC.ts      camera, peer connection, signalling, controls
|           +-- helpers.ts        timeAgo(), getErrorMessage(), small utilities
|           +-- utils.ts          cn() class-name merge helper for Tailwind
|
+-- memory/
|   +-- SPEC.md                   the living specification of the product
|   +-- test_credentials.md       working demo logins
+-- scripts/                      the generators for your two learning documents
```

Why the folders are shaped like this

One router file per feature area

A single 3,000-line server.py is unreadable and impossible to work on with other people. Seventeen files of roughly 250 lines each means a bug in search physically cannot break messaging, and two developers can work in parallel without colliding.

lib/ on both sides means "shared machinery"

Anything used by more than one feature lives in lib/. The database handle, authentication, the fetch layer, the types. If you find yourself copying a function into a second file, it belongs in lib/.

pages/ vs components/

A **page** is a whole screen with a URL. A **component** is a reusable piece used by pages. PostCard is a component because Feed, Profile, Dashboard and Search all render posts; writing it four times would guarantee four different behaviours.

models/schemas.py mirrors lib/types.ts

These two files are the contract between the two programs. Part 6 explains why keeping them in sync is a daily discipline rather than something a tool does for you.

PART 5 — Database architecture

First, what a database actually is

A database is a separate program whose only job is to store data safely and find it again quickly. Your backend talks to it over a connection, the same way your frontend talks to your backend. It keeps data on disk, so a restart loses nothing.

Tables and rows vs collections and documents

Traditional (relational, SQL) databases like PostgreSQL store **tables**. A table has fixed **columns** and each record is a **row**:

```
TABLE users
+--------------------------------------+------------------+---------+
| id                                   | email            | college |
+--------------------------------------+------------------+---------+
| 8f2c...                              | karan@...        | LPU     |   <- a ROW
+--------------------------------------+------------------+---------+
    ^ COLUMN                              ^ COLUMN           ^ COLUMN
```

MongoDB stores **collections** of **documents**. A document is a JSON-like object, and documents in one collection do not have to be identical:

A real document shape from your users collection (values shortened)

```
// one document in the "users" collection
{
  "id": "8f2c...",
  "email": "karan@unisphere.edu",
  "password_hash": "$2b$12$....",
  "full_name": "Karan Kumar",
  "college": "Lovely Professional University",
  "degree": "B.Tech CSE",
  "current_year": "2nd Year",
  "skills": ["Python", "C++", "Machine Learning", "OpenCV"],
  "role": "student",
  "reputation_score": 5.0,
  "connections_count": 12,
  "privacy": { "who_can_message": "everyone", "show_email": false }
}
```

Two things to notice. A field can hold a **list** (skills) or a whole **nested object** (privacy) — a relational table cannot do that without extra tables. And **password_hash** is stored, never the password.

Keys: how records point at each other

- **Primary key** — the field that uniquely identifies a record. Here it is `id`, a UUID string like `8f2c9a1e-....`
- **Foreign key** — a field holding another record's primary key, to express a relationship. `post.author_id` holds a user's id.
- **Why UUID strings and not MongoDB's built-in _id?** Mongo's ObjectId is a special binary type that is not JSON-serialisable, so it leaks into API responses as a crash. Every collection here therefore carries its own string `id`, with a unique index on it.

The three kinds of relationship

```
ONE-TO-ONE      one user has exactly one availability record
                users.id  <---->  availabilities.user_id   (unique index)

ONE-TO-MANY     one user writes many posts
                users.id  <----<  posts.author_id

MANY-TO-MANY    many users like many posts
                users.id  >----  post_likes  ----<  posts.id
                                 (a JOIN collection: one doc per pair)
```

Many-to-many always needs a third collection in the middle, called a join (or junction) collection. That is exactly what post_likes, post_bookmarks, connections, project_requests, college_follows and blocks are.

Why likes are not just an array inside the post

The tempting beginner design is `post.liked_by = [userId, userId, ...]`. It breaks in three ways: the document grows without limit (Mongo caps a document at 16MB), two simultaneous likes can overwrite each other, and "which posts did I like?" requires scanning every post. A join collection with a unique index on (post_id, user_id) fixes all three.

Your 22 collections

Collection	Holds	
users	accounts, student details, hashed password, reputation, privacy	primary entity
connections	one doc per connection request	requester_id + recipient_id -> users
posts	feed items with denormalised author info	author_id -> users
post_likes	one doc per (post, user) like	join: posts x users, unique
post_comments	comments on posts	post_id -> posts, author -> users
post_bookmarks	saved posts	join: posts x users, unique
stories	24h stories with expires_at, views, reactions	author -> users
conversations	a chat between two users + unread counts	participants[] -> users
messages	the individual messages	conversation_id -> conversations
projects	collaboration projects, tech, roles, links	owner_id -> users
project_requests	join requests with a status	project_id + applicant_id
availabilities	one weekly schedule per user	user_id -> users, unique
meetings	a concrete booking of one slot	host_id + guest_id -> users
colleges	community hubs	name unique
college_follows	which student follows which college	join
reputation_reviews	5-dimension endorsements	(reviewer_id, target_user_id) unique
notifications	per-user notification feed	user_id + actor_id -> users
reports	moderation reports for any target type	target_type + target_id
blocks	who blocked whom	(blocker_id, blocked_user_id) unique
meet_sessions	Meet Someone queue + match + signalling	the two matched users
call_sessions	1-to-1 call state + offer/answer/ICE	caller_id + recipient_id
post_shares / misc	supporting records	-

Indexes: the single biggest performance idea in databases

Without an index, finding a record means reading *every* document in the collection. With 1,000 posts that is invisible; with 1,000,000 it is a timeout. An index is a sorted lookup structure, exactly like the index at the back of a textbook: instead of reading the whole book to find "photosynthesis", you look it up and jump straight to page 412.

Your project declares every index in one place and applies them automatically when the server boots, so a fresh database is always correctly indexed:

```
# backend/lib/db.py  (excerpt)
INDEXES: dict[str, list[IndexModel]] = {
    "users": [
        IndexModel([("id", ASCENDING)],    name="user_id_unique",    unique=True),
        IndexModel([("email", ASCENDING)], name="user_email_unique", unique=True),
        IndexModel([("college", ASCENDING)], name="user_college"),
    ],
    "post_likes": [
        IndexModel([("post_id", ASCENDING), ("user_id", ASCENDING)],
                   name="like_unique", unique=True),
    ],
    ...
}

async def ensure_indexes() -> None:
    for collection, models in INDEXES.items():
        for model in models:
            await db[collection].create_indexes([model])

# backend/server.py -- run once at startup, not per request
@asynccontextmanager
async def lifespan(app: FastAPI):
    app.state.index_task = asyncio.create_task(ensure_indexes())
    yield
    client.close()
```

unique=True is a constraint, and constraints are protection

A **constraint** is a rule the database itself enforces. `unique=True` on (post_id, user_id) means the database will physically refuse a second like from the same user on the same post — even if your Python code has a bug, even if two requests arrive in the same millisecond. Application checks can be raced; database constraints cannot.

> *Honest critique, as promised: this project uses MongoDB, while your brief asked for PostgreSQL. The schema was still designed relationally — separate collections, join collections, foreign-key-style ids, unique constraints — which is why it stays clean. The real trade-off is that MongoDB cannot enforce a foreign key: if a user is deleted, the code must clean up their posts, and nothing stops an orphaned author_id. PostgreSQL would enforce that for you. This is a genuine design difference, not a bug, and it is worth understanding before you choose a database for your next project.*

PART 6 — How frontend, backend and database communicate

Step 1: the frontend always uses relative URLs

Frontend code never writes `http://localhost:8001`. It writes `/api/posts`. In development, Vite forwards anything starting with /api to the backend on port 8001. In production, one server serves both. Because the code says "/api", it works unchanged in both places.

```
  DEVELOPMENT                              PRODUCTION
  browser -> :3000 (Vite)                  browser -> one origin
               |  proxy /api                           |
               v                                       v
             :8001 (FastAPI)                         FastAPI
  frontend code is IDENTICAL in both cases
```

Step 2: every request goes through one function

There is exactly one `fetch()` call in the whole frontend, inside `lib/api.ts`. Everything else uses the five helpers it exports.

```
// frontend/src/lib/api.ts  (abridged, real code)
const BASE = "/api";

export class ApiError extends Error {
  status: number; body: unknown;
  constructor(status: number, body: unknown) {
    super(`request failed with ${status}`);
    this.name = "ApiError"; this.status = status; this.body = body;
  }
}

async function request<T>(method: string, path: string, body?: unknown): Promise<T> {
  const token = localStorage.getItem("unisphere_auth_token");
  const headers: Record<string, string> = {};
  if (body != null) headers["Content-Type"] = "application/json";
  if (token) headers["Authorization"] = `Bearer ${token}`;

  const res = await fetch(`${BASE}${path}`, {
    method, headers, credentials: "include",
    body: body == null ? undefined : JSON.stringify(body),
  });

  if (!res.ok) {                                  // fetch does NOT throw on 404/500
    const errBody = await res.json().catch(() => null);
    throw new ApiError(res.status, errBody);
  }
  if (res.status === 204) return undefined as T;  // 204 = success, no content
  return (await res.json()) as T;
}

export const apiGet    = <T>(p: string) => request<T>("GET", p);
export const apiPost   = <T>(p: string, b?: unknown) => request<T>("POST", p, b);
export const apiPut    = <T>(p: string, b?: unknown) => request<T>("PUT", p, b);
export const apiPatch  = <T>(p: string, b?: unknown) => request<T>("PATCH", p, b);
export const apiDelete = <T>(p: string) => request<T>("DELETE", p);
```

The five jobs this one function does for the whole app

- Prefixes /api so no call site can forget it.

- Attaches the login token automatically — no page has to remember authentication.
- Sets the JSON content type only when there is a body.
- Converts any non-2xx response into a thrown ApiError carrying .status and .body, so the UI can react differently to 403 (not allowed) and 422 (invalid input).
- Handles 204 No Content, which has no JSON body at all — parsing it would crash.

Critical beginner trap: fetch() only rejects on network failure. A 404 or a 500 resolves normally, so code without an res.ok check happily continues with an error page as its data. Solving it once, here, is why no page in this app has that bug.

Step 3: the contract — Pydantic model on one side, TypeScript interface on the other

There is no magic bridge between Python and TypeScript. Python declares the response shape; TypeScript declares what it believes it will receive. Nothing checks that the two agree. So the rule in this project is: **change one, change the other in the same edit.**

```
# backend/models/schemas.py
class PostResponse(BaseModel):
    id: str
    author_id: str
    author_name: str
    author_college: str
    content: str
    category: str = "general"
    tags: List[str] = Field(default_factory=list)
    likes_count: int = 0
    comments_count: int = 0
    is_liked_by_me: bool = False
    is_bookmarked_by_me: bool = False
    created_at: datetime
```

```
// frontend/src/lib/types.ts  -- the mirror, field for field
export interface PostResponse {
  id: string;
  author_id: string;
  author_name: string;
  author_college: string;
  content: string;
  category: string;
  tags: string[];
  likes_count: number;
  comments_count: number;
  is_liked_by_me: boolean;
  is_bookmarked_by_me: boolean;
  created_at: string;      // JSON has no Date type -> arrives as a string
}
```

Note `created_at`: Python has a datetime, JSON does not. Over the wire it becomes a string, so the TypeScript type is `string`, and `helpers.ts` turns it into "3 hours ago" for display. Typing it as Date would be a lie the compiler would believe.

The safety net

The only command that detects drift between those two files

```
cd /app/frontend && yarn typecheck
```

Vite *transpiles* TypeScript — it strips the types and runs the JavaScript — but it never checks them. So a mismatch is invisible in the browser until a value renders as undefined. yarn typecheck is the gate.

Step 4: TanStack Query owns reads and writes

Real code from frontend/src/components/PostCard.tsx

```
// reading: PostCard's comment list
const { data: comments, isLoading } = useQuery({
  queryKey: ["comments", post.id],
  queryFn: () => apiGet<CommentResponse[]>(`/posts/${post.id}/comments`),
});

// writing: the like button
const likeMutation = useMutation({
  mutationFn: () => apiPost<{ liked: boolean; likes_count: number }>(
    `/posts/${post.id}/like`),
  onSuccess: invalidate,
});

const invalidate = () => {
  void queryClient.invalidateQueries({ queryKey: ["posts"] });
  void queryClient.invalidateQueries({ queryKey: ["bookmarked-posts"] });
};
```

- **queryKey** is the cache address. Two components asking for ["posts"] share one request and one cached result — not two network calls.
- **queryFn** is how to fetch it. Note it calls apiGet, never fetch.
- **useMutation** is for anything that changes data. It gives you isPending and error for free.
- **invalidateQueries** marks cached data stale, so Query refetches and every component showing posts updates itself. You never manually tell the feed to refresh.

Step 5: the backend decides, then the database stores

```
# backend/routers/posts_router.py -- the whole authorisation idea in 8 lines
@router.delete("/{post_id}")
async def delete_post(post_id: str,
                      current_user: dict = Depends(get_current_user_required)):
    post = await db.posts.find_one({"id": post_id})
    if not post:
        raise HTTPException(status_code=404, detail="Post not found")
    if post["author_id"] != current_user["id"] and current_user.get("role") != "admin":
        raise HTTPException(status_code=403, detail="You can only delete your own posts.")

    await db.posts.delete_one({"id": post_id})
    await db.post_likes.delete_many({"post_id": post_id})       # clean up children
    await db.post_comments.delete_many({"post_id": post_id})
    await db.post_bookmarks.delete_many({"post_id": post_id})
    return {"message": "Post deleted successfully"}
```

Read that carefully, because it contains four separate professional habits:

- **Authentication** (who are you?) is handled by Depends(get_current_user_required) before the function body even runs.
- **Authorisation** (may you do this?) is a separate, explicit check on ownership. Being logged in is not permission.
- **Correct status codes**: 404 for missing, 403 for forbidden, so the frontend can respond meaningfully.

- **Manual cascade**: because MongoDB has no foreign keys, the code deletes the post's likes, comments and bookmarks itself. Forget this and you accumulate orphaned rows forever — exactly the trade-off flagged at the end of Part 5.

PART 7 — Your learning roadmap

This roadmap contains only what **this** project actually needs, plus the prerequisites you need before each step. No Next.js, no GraphQL, no Kubernetes — none of them appear in your code, so learning them now would be noise.

Realistic pace, studying seriously a few hours a day: **LEVEL 0-4 in about a month, LEVEL 5-8 in a second month, LEVEL 9-12 after that.** Do not rush. Each level's exercise matters more than the reading.

LEVEL 0 — How the web works

Priority: ***MUST UNDERSTAND NOW***

- Client and server; what a request and a response are
- URLs, HTTP methods, status codes (200, 401, 403, 404, 422, 500)
- What the browser does with HTML, CSS and JavaScript

Files in your project: `Nothing to open yet — Part 8 of this handbook is this level.`

Exercise: Open your app, press F12, use the Network tab, click Like, and read the request that appears.

LEVEL 1 — HTML and CSS fundamentals

Priority: ***MUST UNDERSTAND NOW***

- Elements, attributes, nesting, forms and inputs
- The box model: margin, border, padding, content
- Flexbox basics; what 'responsive' means and how a breakpoint works

Files in your project: `frontend/index.html, frontend/src/index.css`

Exercise: Change the page title and meta description in index.html; change one colour token in index.css and watch the whole app re-theme.

LEVEL 2 — JavaScript fundamentals

Priority: ***MUST UNDERSTAND NOW***

- Variables, functions, arrays, objects, if/else, loops
- Array methods: map, filter, find, reduce
- async / await and Promises — essential, because every API call uses them
- JSON: what it is and why it is how programs exchange data

Files in your project: `frontend/src/lib/helpers.ts is the gentlest real file in the project.`

Exercise: Read timeAgo() in helpers.ts and explain out loud what every line does.

LEVEL 3 — TypeScript basics

*Priority: **MUST UNDERSTAND NOW***

- Typing variables and function parameters
- interface and type; optional fields with ?; union types like 'image' | 'video'
- Generics, just enough to read apiGet(...)

Files in your project: `frontend/src/lib/types.ts`

Exercise: Add a field to a TS interface that the backend does not send, use it in a component, and watch yarn typecheck stay silent — then see it be undefined in the browser. That lesson sticks.

LEVEL 4 — React fundamentals

*Priority: **MUST UNDERSTAND NOW***

- Components and props; JSX
- useState; why you never mutate state directly
- Events, forms and controlled inputs
- Conditional rendering and rendering lists with a key
- useEffect and cleanup — and why you should not fetch in it here
- Custom hooks and React context

Files in your project: `components/PostCard.tsx (props, state, events), lib/auth-context.tsx (context), pages/Feed.tsx`

Exercise: Add a 'Copy link' button to PostCard that copies the post URL and shows a toast.

LEVEL 5 — Frontend architecture

*Priority: **SHOULD UNDERSTAND SOON***

- Routing and protected routes
- Server state vs UI state, and why TanStack Query exists
- queryKey design and invalidation
- Loading, error and empty states as first-class UI
- Code splitting with lazy() and Suspense

Files in your project: `App.tsx, lib/queryClient.ts, components/States.tsx, components/AppShell.tsx`

Exercise: Add a /bookmarks page: new route, lazy import, useQuery on ['bookmarked-posts'], and a real empty state.

LEVEL 6 — Backend fundamentals

*Priority: **MUST UNDERSTAND NOW***

- What a server process is; ports; localhost
- Routes, handlers, path parameters and query parameters
- Request body vs query string; JSON in and out
- Validation with Pydantic; error responses and status codes
- Dependency injection — what Depends(...) really does

Files in your project: `server.py, routers/posts_router.py, models/schemas.py`

Exercise: Add GET /api/posts/stats returning total posts and posts per category. Register it on the posts router, not on app.

LEVEL 7 — Databases

*Priority: **MUST UNDERSTAND NOW***

- Collections, documents, primary keys, foreign keys
- One-to-one, one-to-many, many-to-many and join collections
- Queries, filters, sorting, skip/limit pagination
- Indexes and unique constraints; denormalised counters
- Enough SQL to understand what a relational database would do differently

Files in your project: `lib/db.py (the index plan), any router's queries, seed.py`

Exercise: Add an index for a filter you use, then add a query that uses it. Then explain why post_likes is a separate collection.

LEVEL 8 — Authentication and authorisation

*Priority: **MUST UNDERSTAND NOW***

- Hashing vs encryption; why bcrypt is deliberately slow
- What a JWT contains and what it does not protect
- Bearer tokens vs cookies; httponly; localStorage trade-offs
- Authentication vs authorisation; ownership checks
- Protected routes on the frontend are UX, not security

Files in your project: `lib/auth.py, routers/auth_router.py, lib/auth-context.tsx, App.tsx (Protected)`

Exercise: Add a rule that only connected users may message each other — enforced in the backend, then reflected in the UI.

LEVEL 9 — Real-time features

*Priority: **SHOULD UNDERSTAND SOON***

- Why request/response is a poor fit for chat
- Polling vs long polling vs WebSockets; the trade-offs
- Events, connection lifecycle, presence, typing indicators
- Notifications as stored records plus a live signal

Files in your project: `routers/messages_router.py, pages/Messages.tsx, the polling in lib/useWebRTC.ts`

Exercise: Measure it: how long after Maya sends a message does Karan see it? Then write down what a WebSocket would change.

LEVEL 10 — Video calling with WebRTC

*Priority: **CAN TREAT AS A LIBRARY (for now)***

- getUserMedia: asking for camera and microphone permission
- What a peer connection is, and why media avoids your server
- SDP, offer and answer — the two sides describing their capabilities
- ICE candidates, STUN and TURN — finding a route through NAT
- Signalling: your server's only job in a call
- Track control: mute, camera off, screen share, and full teardown

Files in your project: `lib/useWebRTC.ts, pages/CallRoom.tsx, routers/calls_router.py, routers/meet_router.py`

Exercise: Do not rebuild this yet. Instead: add a call timer, and make sure ending a call turns the camera light off. Cleanup is the real lesson.

LEVEL 11 — Security

*Priority: **MUST UNDERSTAND NOW (alongside LEVEL 8)***

- Never trust the client; validate everything server-side
- Injection attacks; why never to build queries from raw user strings
- XSS and why React escaping protects you by default
- CSRF, and how Bearer tokens change the picture
- Rate limiting, file-upload validation, secrets in environment variables
- Privacy as a feature: blocking, reporting, visibility controls

Files in your project: `lib/auth.py, routers/safety_router.py, routers/admin_router.py, backend/.env`

Exercise: Try to break your own app: call a protected endpoint with no token, then with another user's post id. Both must be refused.

LEVEL 12 — Git, workflow and deployment

Priority: ***SHOULD UNDERSTAND SOON***

- Repository, commit, branch, merge, pull request; .gitignore
- Environment variables and dev vs production configuration
- What a production build is and why it differs from dev
- Deployment shapes: one host for both, or split frontend/backend
- Logs and monitoring — how you find out something broke

Files in your project: `backend/.env, frontend/vite.config.ts, backend/requirements.txt, frontend/package.json`

Exercise: Write the deployment plan for this app in your own words, including which external services production needs (TURN, email, file storage).

How to classify anything you meet in this codebase

Label	Meaning	
MUST UNDERSTAND NOW	Everything else stands on it	HTTP, React basics, routes, Pydantic, queries
SHOULD UNDERSTAND SOON	Important, can wait weeks	TanStack Query internals, real-time design, G
CAN TREAT AS A LIBRARY	Use correctly now, build later	WebRTC internals, shadcn/ui primitives, bcryp

Using something correctly long before you could implement it is not cheating — it is how every professional works. Nobody writes their own bcrypt.

PART 8 — Lesson 1: how the web actually works

One lesson at a time. This is LEVEL 0. Do not skip it because it sounds basic: almost every confusing thing later ("why is my token undefined?", "why is it 401?") is a gap in this lesson.

1. Concept

The web runs on one exchange, repeated forever: a **client** sends a **request**, a **server** sends back a **response**. That is it. Every page load, every login, every like is one of these.

A request has four parts worth knowing:

- **Method** — the verb. GET = read, POST = create/do, PUT/PATCH = update, DELETE = remove.
- **URL** — the address, e.g. /api/posts/abc123/like.
- **Headers** — metadata. This is where your login token travels.
- **Body** — the data you are sending (only for POST/PUT/PATCH). Usually JSON.

A response has three:

- **Status code** — a number saying what happened. 200 OK. 401 not logged in. 403 logged in but not allowed. 404 not found. 422 your data was invalid. 500 the server broke.
- **Headers** — metadata about the response.
- **Body** — the data, usually JSON.

The one thing that surprises everybody

HTTP is stateless. The server does not remember you between requests. Your second request arrives as a total stranger. That single fact is the entire reason tokens, cookies and sessions exist — they are how each request re-proves who you are. Keep this in mind and authentication will feel obvious later instead of magical.

2. Why does this exist?

The web had to work between machines that know nothing about each other, run different software, and may disconnect at any moment. Statelessness is what makes that survivable: any server can answer any request, so a service can run on a thousand machines behind one address, and one crashing loses nothing but the request in flight.

3. A tiny example (nothing to do with this project)

Ordering coffee.

```
  YOU (client)                                  BARISTA (server)
  "One flat white, oat milk"   -- request -->
                                                makes it
  cup                          <-- response --

  Next morning you return. The barista does not remember you.
  You state your order again -> STATELESS.
  Give you a loyalty card, and each visit you present the card
  -> that card is a TOKEN. That is exactly what a JWT is.
```

4. How this project uses it

Every single feature of Unisphere is this exchange. Let us look at one real request, in three layers: what the frontend sends, what the backend does, what comes back.

The complete exchange behind one tap of the heart button

```
POST /api/posts/abc123/like
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
(no body)

--- server does its work ---

200 OK
{"liked": true, "likes_count": 8}
```

5. Code walkthrough

The browser side — components/PostCard.tsx

```
const likeMutation = useMutation({
  mutationFn: () => apiPost<{ liked: boolean; likes_count: number }>(
    `/posts/${post.id}/like`),
  onSuccess: invalidate,
});

<button
  data-testid={`post-like-btn-${post.id}`}
  onClick={() => likeMutation.mutate()}
>
  <Heart ... />
  <span data-testid={`post-like-count-${post.id}`}>{post.likes_count}</span>
</button>
```

- `onClick` is a browser event: the user tapped, so run this function.
- `likeMutation.mutate()` starts the request. Nothing about URLs, tokens or JSON appears here — the button's job is to say what happened, not how to talk to a server.
- `apiPost<{liked, likes_count}>` declares the response shape, so TypeScript will catch you if you read a field the backend never sends.
- `{post.likes_count}` renders the number from data. When the data changes, React redraws it. You never write "set the text of that span to 8".
- `data-testid` is a stable hook for automated tests, so tests do not break when the styling changes.

The transport — lib/api.ts

```
const token = localStorage.getItem("unisphere_auth_token");
if (token) headers["Authorization"] = `Bearer ${token}`;
```

```
const res = await fetch(`${BASE}${path}`, { method, headers, credentials: "include", ... });
if (!res.ok) throw new ApiError(res.status, await res.json().catch(() => null));
return (await res.json()) as T;
```

This is where the abstract "headers" from the diagram become real. The token was saved at login; every request re-presents it. This is the loyalty card from the coffee example.

The server side — routers/posts_router.py

The real handler, unabridged

```
@router.post("/{post_id}/like")
async def toggle_like(post_id: str,
                      current_user: dict = Depends(get_current_user_required)):
    post = await db.posts.find_one({"id": post_id})
    if not post:
        raise HTTPException(status_code=404, detail="Post not found")

    existing_like = await db.post_likes.find_one(
        {"post_id": post_id, "user_id": current_user["id"]})

    if existing_like:                                  # already liked -> unlike
        await db.post_likes.delete_one(
            {"post_id": post_id, "user_id": current_user["id"]})
        await db.posts.update_one({"id": post_id}, {"$inc": {"likes_count": -1}})
        return {"liked": False, "likes_count": max(0, post.get("likes_count", 1) - 1)}

    await db.post_likes.insert_one({                   # not liked -> like
        "post_id": post_id, "user_id": current_user["id"],
        "created_at": datetime.utcnow(),
    })
    await db.posts.update_one({"id": post_id}, {"$inc": {"likes_count": 1}})

    if post["author_id"] != current_user["id"]:        # tell the author
        await db.notifications.insert_one({
            "id": str(uuid.uuid4()), "user_id": post["author_id"],
            "actor_id": current_user["id"], "type": "post_like",
            "message": f"{current_user['full_name']} liked your post.",
            "link": f"/feed?post={post_id}", "is_read": False,
            "created_at": datetime.utcnow(),
        })

    return {"liked": True, "likes_count": post.get("likes_count", 0) + 1}
```

Line by line, the parts that matter:

- `@router.post("/{post_id}/like")` — a decorator that says "when a POST arrives at this URL, run this function". The `{post_id}` in braces is a **path parameter**: whatever appears there is handed to the function as an argument.
- `Depends(get_current_user_required)` — FastAPI runs that function first. It reads the token, verifies the signature, loads the user, refuses suspended accounts, and raises 401 if anything fails. If it raises, your handler never runs at all. This is **middleware-style authentication**, expressed as a dependency.
- `await` on every database call — the server hands control back to other users while the database is working, instead of freezing.
- `find_one` then `404` — never assume the record exists. Ids come from the internet; the internet lies.
- `{"$inc": {"likes_count": 1}}` — "increase this field by one", performed *inside* the database. Reading 7, adding 1 and writing 8 in Python would lose a like if two people clicked at the same moment. $inc cannot.

- The notification is written only when the liker is not the author — a small product detail that separates a real app from a demo.
- `toggle`, not "add" — one endpoint handles like and unlike, because the truth lives in the database, not in what the button thinks its state is.

6. What happens internally, step by step

```
 1  tap                    browser fires a click event
 2  onClick                React runs likeMutation.mutate()
 3  useMutation            sets isPending, calls mutationFn
 4  apiPost                builds URL + Authorization header
 5  fetch                  the actual network request leaves the browser
 6  Vite proxy (dev)       /api/... forwarded to port 8001
 7  uvicorn                accepts the connection, hands it to FastAPI
 8  FastAPI router         matches POST /api/posts/{post_id}/like
 9  Depends(...)           JWT decoded -> user loaded -> or 401 and STOP
10  handler                post exists? -> or 404 and STOP
11  handler                already liked? -> branch
12  MongoDB                write post_likes  +  $inc likes_count
13  MongoDB                write notification for the author
14  FastAPI                serialises the return value to JSON, 200
15  fetch resolves         api.ts checks res.ok, parses JSON
16  onSuccess              invalidateQueries(['posts'])
17  TanStack Query         refetches GET /api/posts (repeat 4-15 for a GET)
18  React                  re-renders PostCard with new data
19  user sees              filled heart, count 8
```

Nineteen steps for one tap. This is why architecture matters: each step is small, replaceable and testable on its own. And it is why **reading the Network tab** is the most valuable debugging habit you can build — it shows you steps 5 to 15 directly.

7. Common beginner mistakes

- **Thinking the frontend "has" the data.** It has a cached copy. The database is the truth.
- **Assuming fetch throws on errors.** It does not. A 500 resolves normally; you must check res.ok. This is why the whole app funnels through api.ts.
- **Confusing 401 and 403.** 401 = I do not know who you are. 403 = I know exactly who you are, and you may not do this. Different fixes entirely.
- **Believing hidden buttons are security.** Hiding the admin link is UX. Someone can still call your API directly with curl. Security lives on the server, always.
- **Updating the count in the browser instead of refetching.** Works until two devices, or one failed request, make your number a fiction.
- **Reading a count as truth without knowing it is denormalised.** likes_count is a cached number kept in step by $inc; post_likes is the real record.

8. Mini task (do this before Lesson 2)

- Open the app, log in as `karan@unisphere.edu / Student@123456`.
- Press F12, open the **Network** tab, filter to Fetch/XHR.
- Click the like button on a post. Find the request that appears.

- Write down, from what you see: the method, the full URL, the status code, the response body, and the value of the Authorization header.
- Now click like again (unlike) and compare the response body. Explain why it differs.
- Reload the page and count how many requests fire before the feed appears. Name what each one is fetching.

If you can do this, you can debug. Most "my app is broken" questions are answered by the Network tab in thirty seconds.

9. Check your understanding

Answer these in your own words. Do not look them up — I want your mental model, mistakes included, so I can correct the model rather than the sentence.

- 1. In one sentence each: what is a client, what is a server, and which one can be trusted?
- 2. What does it mean that HTTP is stateless, and what does this project do about it?
- 3. Explain the difference between 401 and 403 using a Unisphere example.
- 4. Why does the like handler use `$inc` instead of reading likes_count, adding one, and writing it back?
- 5. After liking a post, why does the frontend refetch the posts instead of just adding one to the number on screen?

Send me your answers and we will do **Lesson 2: HTML, CSS and how one file re-themes your entire app**. We do not move on until Lesson 1 is solid.

APPENDIX A — API reference

Every endpoint lives under `/api`, registered on `api_router` in server.py. This is a representative map, not an exhaustive list of all 100+ routes — open any router file to see the rest.

Method	Endpoint	Auth	Purpose
GET	/api/	no	health check + version
POST	/api/auth/signup	no	create an account, return user + token
POST	/api/auth/login	no	verify password, return user + token
POST	/api/auth/logout	yes	clear the session cookie
GET	/api/auth/me	yes	who am I? used to restore a session on reload
GET	/api/users	yes	discovery with filters (college, degree, year, skills...)
GET	/api/users/{id}	optional	public profile
PUT	/api/users/me	yes	edit my own profile
GET	/api/posts	optional	feed, filtered + paginated, blocked users removed
POST	/api/posts	yes	create a post
DELETE	/api/posts/{id}	yes	delete own post (or admin)
POST	/api/posts/{id}/like	yes	toggle like + notify author
POST	/api/posts/{id}/comments	yes	add a comment
GET	/api/posts/{id}/comments	optional	list comments
POST	/api/posts/{id}/bookmark	yes	toggle bookmark
GET	/api/stories	yes	active (non-expired) stories
POST	/api/stories	yes	publish a 24h story
POST	/api/connections/request	yes	send a connection request
POST	/api/connections/{id}/accept	yes	accept a request
GET	/api/messages/conversations	yes	inbox list with unread counts
POST	/api/messages/send	yes	send a message
GET	/api/projects	yes	browse projects
POST	/api/projects/{id}/request	yes	request to join
GET	/api/meetings	yes	my meetings by status
POST	/api/meetings/book	yes	book an available slot
POST	/api/meet/queue	yes	join the Meet Someone queue, get matched
POST	/api/meet/signal/send	yes	relay an SDP offer/answer or ICE candidate
GET	/api/meet/signal/poll	yes	collect signals waiting for me
POST	/api/calls/start	yes	create a call session (ringing)
GET	/api/notifications	yes	my notification feed
POST	/api/safety/report	yes	report a user/post/comment/message/story
POST	/api/safety/block	yes	block a user
GET	/api/search	yes	global search across four entity types

Method	Endpoint	Purpose	Auth
GET	/api/admin/metrics	admin dashboard counters	admin
POST	/api/admin/users/{id}/suspend	suspend an account	admin

Reading the Auth column

- **no** — anyone may call it. Used only for landing, signup and login.
- **optional** — works signed out, but returns extra fields when signed in (is_liked_by_me, blocked-user filtering). Implemented with get_current_user_optional.
- **yes** — get_current_user_required; 401 without a valid token.
- **admin** — get_current_admin_user; 403 for a logged-in non-admin.

```
# backend/lib/auth.py -- the three gates, real code
async def get_current_user_required(request: Request) -> Dict[str, Any]:
    user = await get_current_user_optional(request)
    if not user:
        raise HTTPException(status_code=401,
            detail="Authentication required. Please log in to proceed.",
            headers={"WWW-Authenticate": "Bearer"})
    return user

async def get_current_admin_user(
        current_user: Dict[str, Any] = Depends(get_current_user_required)):
    if current_user.get("role") != "admin":
        raise HTTPException(status_code=403, detail="Admin privileges required.")
    return current_user
```

APPENDIX B — Nine real user actions, traced end to end

Learn features as journeys, not as files. Each trace below is the real path through your code. Follow one with the files open beside you.

B1 — Registration

```
Signup.tsx form (name, email, password, college, degree, branch,
  year, grad year, city, interests, skills)
  |  useState holds each field as you type (controlled inputs)
  v
submit -> auth-context.signup(data)
  |
  v apiPost('/auth/signup', data)  [lib/api.ts]
  v
POST /api/auth/signup  [auth_router.py]
  |  Pydantic UserSignup validates types + required fields -> 422 if bad
  |  email lowercased and trimmed
  |  db.users.find_one({email}) -> already exists? 400 with a clear message
  |  hash_password(password)   <-- bcrypt; the plain password is never stored
  |  build the user document: role='student', reputation 5.0,
  |    category_reputation (5 dimensions), privacy defaults, avatar fallback
  |  db.users.insert_one(user)
  |  db.availabilities.insert_one(default Mon-Fri weekly slots)
  |  create_access_token({sub: user_id, email, role})
  |  response.set_cookie('session_token', httponly=True, 30 days)
  v
200 { user, token, message }
  |  auth-context stores token in localStorage, sets user state
  |  queryClient.invalidateQueries()  -> every cached screen refetches as "me"
  v
navigate to /dashboard -> AppShell renders -> personalised greeting
```

Two details worth copying into your own projects. First, `password_hash` is created before the insert, and the plain password is popped off the dictionary — there is no code path where it could be written to disk. Second, signup also creates a default availability record, so the Meetings page is never a broken empty screen for a new user. Good products seed their own defaults.

> *Honest note: the token is stored in localStorage and an httponly cookie is set. localStorage is readable by any JavaScript on the page, so it is vulnerable to XSS; the httponly cookie is not. A hardened production version would use the cookie alone. The cookie is already there, which makes that a small change — but it is a real trade-off you should be able to explain in an interview.*

B2 — Login and staying logged in

```
Login.tsx -> auth-context.login(email, password)
  v POST /api/auth/login
  |  find user by email        -> not found? generic error (do NOT reveal
  |                               whether the email exists: that is a
  |                               user-enumeration leak)
  |  verify_password(plain, stored_hash)   <-- bcrypt re-hashes and compares
  |  suspended? refuse
  |  create_access_token(...)
  v 200 { user, token }
  |  localStorage.setItem('unisphere_auth_token', token)
```

```
   v
 LATER: the user reloads the page. React state is gone.
   AuthProvider's useEffect runs once on mount:
     token in localStorage?  no  -> isLoading=false, show public UI
                             yes -> GET /api/auth/me
                                      ok    -> setUser(me)
                                      fails -> remove token, setUser(null)
```

```
// frontend/src/lib/auth-context.tsx -- session restore, real code
useEffect(() => {
  const bootstrap = async () => {
    const token = localStorage.getItem(TOKEN_KEY);
    if (!token) { setIsLoading(false); return; }
    try {
      const me = await apiGet<UserResponse>("/auth/me");
      setUser(me);
    } catch {
      localStorage.removeItem(TOKEN_KEY);   // stale/expired token: clean up
      setUser(null);
    } finally {
      setIsLoading(false);
    }
  };
  void bootstrap();
}, []);
```

Why isLoading exists. On the very first paint the app does not yet know whether you are logged in. Without this flag, protected screens would flash the login page and then redirect — the classic auth flicker. The empty dependency array `[]` means "run once when mounted", not on every render.

Why /auth/me exists at all. The token contains a user id, but the user's name, avatar, college and reputation may have changed since it was issued. So the app asks the server for the current truth rather than trusting old data baked into the token.

B3 — Creating a post

```
 Feed.tsx composer: content, category, optional media_url
   v useMutation -> apiPost('/posts', { content, category, tags, media_url })
   v POST /api/posts   [posts_router.py]
   |  Depends(get_current_user_required)  -> 401 if not logged in
   |  Pydantic PostCreate validates the body
   |  build the document, stamping author details onto the post:
   |    author_id, author_name, author_avatar, author_college,
   |    author_degree_year, likes_count=0, comments_count=0, created_at
   |  db.posts.insert_one(post)
   v 200 PostResponse
   |  onSuccess -> invalidateQueries(['posts'])
   v feed refetches; the new post is at the top (sorted created_at desc)
```

Why the author's name is copied onto every post

Strict relational thinking says store only `author_id` and look the user up when displaying. But a feed of 50 posts would then need 50 extra lookups (the classic "N+1 query problem"). Copying the display fields onto the post makes the feed a single query.

The cost is real and you should know it: if Karan renames himself, old posts keep the old name until something updates them. This is denormalisation again — **read speed bought with duplication and staleness.** Production systems accept this deliberately and add a background job to refresh copies. Knowing you made the trade is what makes it engineering rather than an accident.

B4 — Liking a post

Fully traced in Part 3 and Part 8. The one-line summary: a join document in `post_likes` plus an atomic `$inc` on the post, plus a notification for the author, then cache invalidation on the client.

B5 — Sending a connection request

```
Discover.tsx / Profile.tsx  [Connect]
  v POST /api/connections/request { recipient_id }
  |  auth required
  |  refuse self-connection
  |  is there already a connection doc for this pair (either direction)?
  |    -> yes: return its state instead of creating a duplicate
  |  privacy: does the recipient allow connection requests?
  |  blocks: has either user blocked the other?
  |  insert { id, requester_id, recipient_id, status:'pending', created_at }
  |  insert a notification for the recipient
  v the button's label is DERIVED from that one document:
      no doc            -> "Connect"
      status=pending    -> "Pending"
      status=accepted   -> "Connected"
  v on accept: status='accepted', both users' connections_count $inc 1,
               notification back to the requester
```

The important design idea: **the three button states are not three fields.** They are one document's status, read from either direction. Storing "is_pending" and "is_connected" separately guarantees they eventually contradict each other.

B6 — Sending a message

```
Messages.tsx -> POST /api/messages/send { recipient_id, content }
  |  find a conversation whose participants contain both users,
  |    or create one
  |  insert the message { conversation_id, sender_id, content, created_at }
  |  update the conversation: last_message, last_message_at,
  |    and $inc the recipient's unread counter
  |  insert a notification
  v the thread refetches (and polls), the message appears
```

Why unread lives on the conversation, not computed from messages. The inbox needs a badge for every conversation. Computing it honestly means scanning each thread's messages on every inbox load. One stored counter turns that into an instant read. Same trade-off as likes_count — you are starting to see that denormalisation is a pattern, not a hack.

> *This is where polling shows its limit: the recipient learns about the message on their next poll, not the instant it is sent. That is the LEVEL 9 upgrade to WebSockets.*

B7 — Creating a project and requesting to join

```
CREATE   POST /api/projects { title, description, category, technologies,
                              required_roles, github_url, demo_url, status }
           -> owner_id = current user; insert

JOIN     POST /api/projects/{id}/request { role, pitch }
           -> project exists? owner is not the applicant?
           -> already requested? (no duplicates)
           -> insert project_requests { project_id, applicant_id,
                                        role, pitch, status:'pending' }
```

```
            -> notify the owner

REVIEW   POST /api/projects/{id}/requests/{req}/accept
            -> ONLY the owner may call this (ownership check)
            -> request.status='accepted'; applicant added to team_members
            -> notify the applicant
```

Note why `project_requests` is its own collection: the owner needs a reviewable queue with history (who asked, for which role, with what pitch, and what was decided). Pushing applicants straight into a members array would throw all of that away and make "accept" and "is a member" the same unrecoverable action.

B8 — Availability and booking a meeting

```
availabilities  = one document per user (unique index on user_id)
                  topics[], timezone, weekly_schedule[
                    { day, active, slots:[{start_time,end_time,is_booked}] }]

meetings        = one document per actual booking
                  { host_id, guest_id, meeting_date, start, end,
                    title, description, status }

BOOK   POST /api/meetings/book { host_id, date, start_time, title, ... }
          -> is that slot in the host's schedule and not booked?
          -> insert the meeting with status 'upcoming'
          -> mark the slot is_booked
          -> notify the host
STATUS upcoming -> completed | cancelled, each with a notification
```

Two collections instead of one is the whole trick. "What do you offer?" and "what is actually booked?" are different questions with different lifetimes. Merge them and you get contradictions like a slot that is both free and taken.

B9 — Meet Someone, and then a real video call

This is the hardest flow in the project, so take it in three stages.

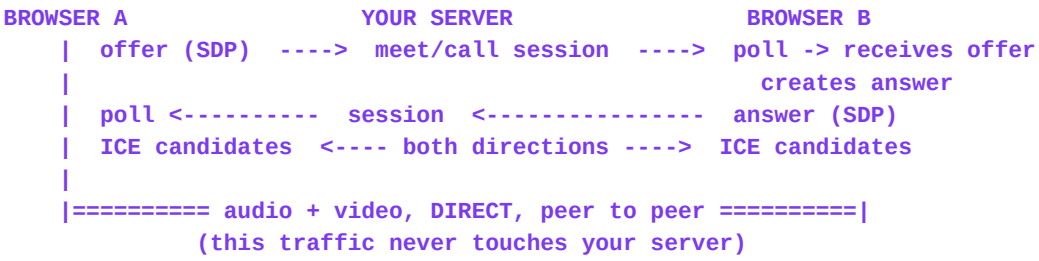
Stage 1 — matching (ordinary backend logic, nothing exotic)

```
MeetSomeone.tsx: user picks intents (new friends / project partners /
  study partners / career / networking / hackathon teammates)
  plus optional filters (same field, different college, same year,
  similar interests, specific skills)
  v POST /api/meet/queue { intents, filters }
  |  write a meet_session with status 'waiting'
  |  look for another compatible waiting session
  |     none found      -> stay waiting; the UI shows "Finding a student..."
  |     found a peer    -> both sessions become status 'matched' and share
  |                        one session id
  v the UI reveals only first name, college, year, interests
    -- never email or contact details (privacy by default)
  v [Start Video] [Start Chat] [Next] [Report] [Block] always available
```

Stage 2 — what a video call needs, in plain language

Two browsers must send video directly to each other. Before they can, they have to agree on codecs and find a network route between them — and neither browser knows the other exists. Your server's *only* job is to pass notes between them. That note-passing is called **signalling**.

- **SDP** (Session Description Protocol) — a text blob describing "here is the video and audio I can send and receive".
- **Offer** — the caller's SDP. **Answer** — the callee's SDP in reply.
- **ICE candidate** — one possible network address/route to reach me. Each side discovers several and sends them over.
- **STUN** — a tiny public service that tells your browser its own public address, because behind a home router it genuinely does not know it.
- **TURN** — a relay used when no direct route can be found (strict corporate or mobile networks). It costs money because it carries the media. **This project has STUN but not TURN**, which is why some networks will fail to connect: a documented production gap, not a bug.



Stage 3 — the code that does it, lib/useWebRTC.ts

Abridged from the real file; the same nine ideas, in order

```
// 1. ask the user for camera + microphone (a permission prompt appears)
if (!navigator.mediaDevices?.getUserMedia) {
  setMediaError("This browser does not support camera access.");
  return;
}
const stream = await navigator.mediaDevices.getUserMedia({ video: true, audio: true });

// 2. create the peer connection, told where to find STUN
const pc = new RTCPeerConnection(ICE_SERVERS);

// 3. put my camera/mic tracks into the connection
stream.getTracks().forEach((t) => pc.addTrack(t, stream));

// 4. whenever the browser discovers a route to me, send it to the peer
pc.onicecandidate = (e) => {
  if (e.candidate) void apiPost(`/${channel}/signal/send`, { ... });
};

// 5. when the peer's media arrives, show it
pc.ontrack = (e) => setRemoteStream(e.streams[0]);

// 6. caller side: describe myself and send the offer
const offer = await pc.createOffer({ offerToReceiveAudio: true,
                                      offerToReceiveVideo: true });
await pc.setLocalDescription(offer);
await apiPost(`/${channel}/signal/send`, { signal_type: "offer", payload: offer });

// 7. keep checking for notes from the peer
const poll = async () => {
  const res = await apiGet<{ signals: Array<{...}> }>(`/${channel}/signal/poll?...`);
  for (const sig of res.signals) {
    if (sig.signal_type === "offer") {
      await pc.setRemoteDescription(new RTCSessionDescription(...));
      const answer = await pc.createAnswer();
      await pc.setLocalDescription(answer);
      await apiPost(`/${channel}/signal/send`, { signal_type: "answer", payload: answer });
```

```
    } else if (sig.signal_type === "answer") {
      await pc.setRemoteDescription(new RTCSessionDescription(...));
    } else if (sig.signal_type === "ice") {
      await pc.addIceCandidate(new RTCIceCandidate(sig.payload.candidate));
    }
  }
};
pollRef.current = window.setInterval(() => void poll(), 2500);
```

The controls are real, not cosmetic

```
// mute / camera off: flip the actual track, do not fake the icon
track.enabled = !track.enabled;

// screen share: swap the outgoing video track on the sender
const screen = await navigator.mediaDevices.getDisplayMedia({ video: true });
const sender = pc.getSenders().find((s) => s.track?.kind === "video");
if (sender) await sender.replaceTrack(screen.getVideoTracks()[0]);
```

> *The most important five lines in the whole file are the cleanup: stop every track, close the peer connection, clear the poll interval. Skip them and the user's camera light stays on after they leave the page — which users, correctly, treat as spyware. A leaked setInterval also keeps signalling for a call that no longer exists.*

The demo fallback, declared openly

If no second human is online, the Meet flow can pair you with an interactive simulation partner so the journey is testable alone. It is labelled as a simulation. This matters ethically and professionally: a stub that pretends to be a real peer is worse than no feature, because it destroys trust in everything else you built.

APPENDIX C — Commands, logins and how to debug

Working logins

Account	Email	Password	
Karan Kumar	karan@unisphere.edu	Student@123456	the main student journey
Maya Lin	maya@stanford.edu	Student@123456	the second side of chat / calls
Aarav Sharma	aarav@mit.edu	Student@123456	projects and connections
Elena Rostova	elena@oxford.edu	Student@123456	cross-college discovery
David Chen	david@berkeley.edu	Student@123456	meetings and reputation
Admin	admin@unisphere.edu	Admin@123456	the /admin dashboard

Test chat, connections and video calls by opening two browser windows (one normal, one incognito) and logging in as two different students. This is how every developer tests a two-person feature.

Commands you will actually use

```
# service state and logs (the first thing to check when something breaks)
sudo supervisorctl status backend frontend
tail -n 50 /var/log/supervisor/backend.err.log
tail -n 50 /var/log/supervisor/frontend.err.log

# restart -- ONLY needed after editing .env or installing a dependency
sudo supervisorctl restart backend frontend

# is a TypeScript / backend contract broken? the one true gate
cd /app/frontend && yarn typecheck

# call the API directly, no browser involved
curl -s http://localhost:8001/api/ | head
curl -s -X POST http://localhost:8001/api/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"email":"karan@unisphere.edu","password":"Student@123456"}'

# prove that protection works: no token must mean 401
curl -s -o /dev/null -w '%{http_code}\n' http://localhost:8001/api/auth/me

# repopulate the demo data
cd /app/backend && python seed.py
```

You do **not** need to restart anything for ordinary code edits: the backend reloads on save (uvicorn --reload) and the frontend hot-reloads (Vite).

How to debug like a developer, in five steps

- **1. Read the actual error.** Not the vibe of it — the words. "undefined is not a function", "401 Unauthorized", "KeyError: 'author_id'" each point somewhere specific.
- **2. Decide which side is broken.** Open the Network tab. If the request never leaves, it is a frontend problem. If it leaves and returns 4xx/5xx, it is a backend problem. This one question cuts your search space in half in ten seconds.

- **3. Reproduce it deliberately.** Find the shortest exact sequence that triggers it. A bug you cannot reproduce on demand cannot be verified as fixed.
- **4. Bisect.** Log or print at the halfway point of the flow. Was the data already wrong there? Move earlier. Still correct? Move later. Roughly ten steps finds a bug in a thousand-line path.
- **5. Fix, then re-run the exact original reproduction.** "Should be fixed" is not a status. If the bug touched both sides, verify both: curl for the API, and a real click for the UI.

Error messages you will meet in this project, decoded

What you see	What it usually means	
401 Unauthorized	no token, expired token, or suspended account	localStorage token; lib/auth.py
403 Forbidden	logged in, but not the owner / not admin	the ownership check in the router
404 Not Found	wrong URL, or the id does not exist	the route path; the id you passed
422 Unprocessable	your JSON body failed Pydantic validation	the model in schemas.py vs what you sen
500 Internal Error	the backend raised an exception	backend.err.log -- read the traceback
undefined on screen	TS interface and Pydantic model disagree	types.ts vs schemas.py; run yarn typeche
query returns []	the filter is wrong, not necessarily the data	print the query; test it directly
CORS error	origin not allowed by the backend	CORS_ORIGINS in backend/.env

Four real bugs from this build, and what each one teaches

- **MongoDB $in with $regex returned nothing.** The query looked perfectly correct and silently matched zero documents; the fix was an $or of individual $regex clauses. *Lesson: an empty result is not proof the data is missing. Suspect the query.*
- **A missing `uuid` import in admin_router.py.** Fine until the first admin write, then a 500. *Lesson: an error inside a rarely-taken branch only appears when that branch runs. Exercise every endpoint at least once.*
- **Icon names that did not exist in lucide-react.** Invisible in the browser, caught by yarn typecheck. *Lesson: the typechecker is a second pair of eyes, not bureaucracy.*
- **Toasts overlapped the notification bell and swallowed clicks.** Fixed by anchoring them bottom-right. *Lesson: layout collisions are invisible in code review and obvious in a screenshot. Look at your app, not only at your code.*

Git and professional workflow, briefly

- **Repository** — the project plus its full history.
- **Commit** — a saved snapshot with a message. Commit small and often; "fixed stuff" is a message your future self will curse.
- **Branch** — a parallel line of work, so an experiment cannot break the working app.
- **Merge / pull request** — bringing a branch back, with a chance for review first.
- **.gitignore** — the list of things never to commit. `.env` and `node_modules` belong here. A secret committed once lives in the history forever, even if you delete the file.
- **Dev vs production** — same code, different configuration, supplied by environment variables. That is why nothing in this project hardcodes a URL, a port or a key.

What production would still need

- **A TURN server** so calls connect on restrictive networks (STUN alone is not enough).
- **Real email** (Resend/SendGrid) for verification and password reset — the flows exist, the delivery does not.
- **Object storage** (S3 or similar) for uploads, with type and size validation, instead of URLs.
- **Rate limiting** on login, signup, reporting and posting, to blunt abuse and brute force.
- **WebSockets** replacing polling for chat and signalling.
- **Secure cookie-only sessions** instead of a token in localStorage.

Your next step is Lesson 1's mini task and its five questions in Part 8. Send me your answers — including the ones you are unsure about — and we will begin Lesson 2. I would rather correct your mental model early than teach you something on top of a crack.